1. 주어진 3가지의 위치벡터를 Algorithm 51, 즉 Angles-only Gauss' method로 구하는 문제이다. Algorithm 51은 site 파일에 해당되고 파일을 실행해보니 다음과 같이 C, r_{del}, r_k에 대한 함수로 표현되어 있음 알 수 있다.

```
편집기 - C:₩Users₩이진진₩Downloads₩우동2_HW4_codes₩우동2_HW4_code2016_수정₩site.m
ex7_2.m   site.m
%
% ----------------------------------------------------------------------
%
%                           function site
%
%  this function finds the position and velocity vectors for a site.  the
%    answer is returned in the geocentric equatorial (ecef) coordinate system.
%    note that the velocity is zero because the coordinate system is fixed to
%    the earth.
%
%  author        : david vallado                  719-573-2600   25 jun 2002
%
%  revisions
%    vallado     - fix velocity vector                            23 jul 2002
%
%  inputs          description                    range / units
%    latgd       - geodetic latitude              -pi/2 to pi/2 rad
%    lon         - longitude of site              -2pi to 2pi rad
%    alt         - altitude                       km
%
%  outputs       :
%    rs          - ecef site position vector      km
%    vs          - ecef site velocity vector      km/s
%
%  locals        :
%    sinlat      - variable containing  sin(lat)  rad
%    temp        - temporary real value
%    rdel        - rdel component of site vector  km
%    rk          - rk component of site vector    km
%    cearth      -
%
%  coupling      :
%    none
%
%  references    :
%    vallado       2001, 404-407, alg 47, ex 7-1
%
% [rs,vs] = site ( latgd,lon,alt );
% ----------------------------------------------------------------------

function [rs,vs] = site ( latgd,lon,alt );

function [rs,vs] = site ( latgd,lon,alt );

        constastro;

        % ------------------------  implementation   ----------------
        sinlat      = sin( latgd );

        % ------  find rdel and rk components of site vector  ---------
        cearth= re / sqrt( 1.0 - ( eccearthsqrd*sinlat*sinlat ) );
        rdel  = ( cearth + alt )*cos( latgd );
        rk    = ( (1.0-eccearthsqrd)*cearth + alt )*sinlat;

        % ---------------  find site position vector  ----------------
        rs(1) = rdel * cos( lon );
        rs(2) = rdel * sin( lon );
        rs(3) = rk;
        rs = rs';

        % ---------------  find site velocity vector  ----------------
        [vs] = [0.0; 0.0; 0.0];
```

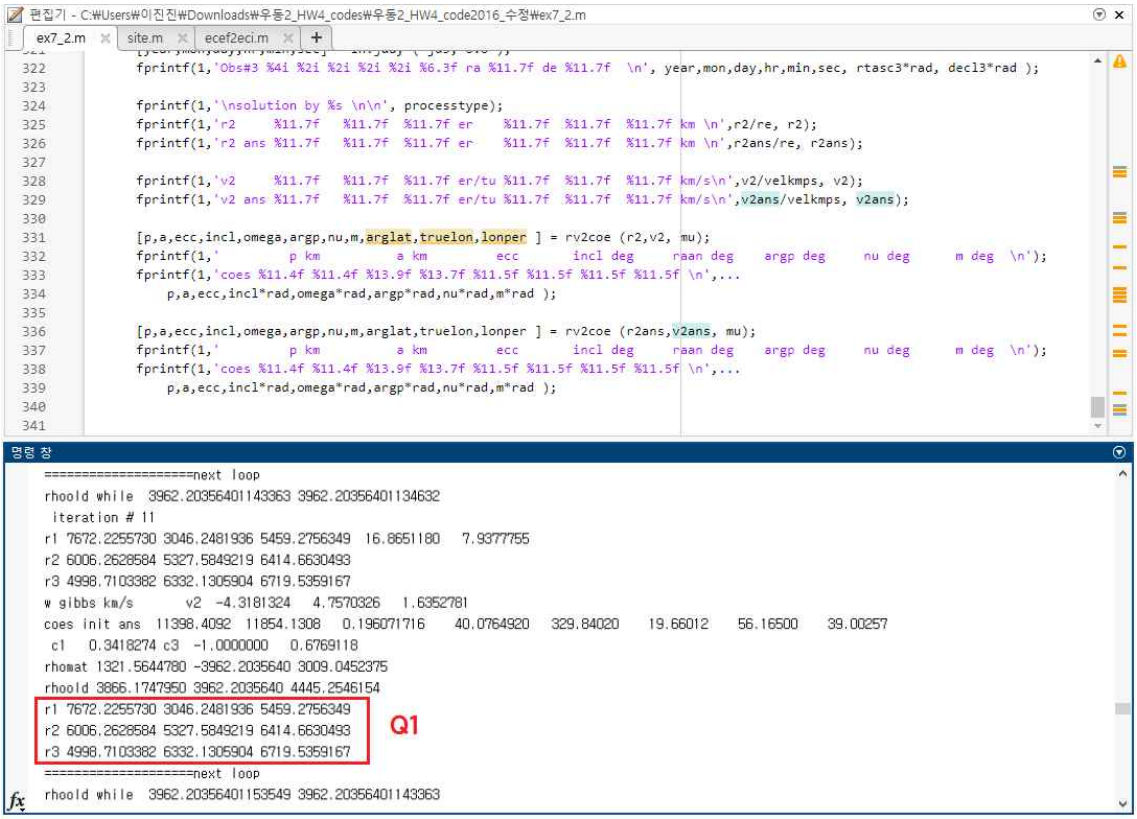
(1) 이를 통해 얻은 값은 IJK좌표계(ECEF)이므로 이것을 ECI좌표계로 변환해야 한다. 이것을 해결하기 위해 ecef2eci 파일의 수치를 좌표계에 맞게 변환시킨 후 실행하면 아래와 같이 $\vec{r}_{site}$의 3방향의 벡터 결과 값을 얻을 수 있다.

```
편집기 - C:₩Users₩이진진₩Downloads₩우동2_HW4_codes₩우동2_HW4_code2016_수정₩ex7_2.m
ex7_2.m   site.m   ecef2eci.m
325        fprintf(1,'r2      %11.7f    %11.7f   %11.7f er     %11.7f   %11.7f   %11.7f km \n

명령 창
lmatii =

    0.9298    0.5114    0.2405
    0.0865    0.6246    0.7672
    0.3578    0.5902    0.5946

rsmat eci  4077.3897186  3979.9966715  3929.4655269 km
rsmat eci  2711.9120862  2852.8315730  2921.9914652 km
rsmat eci  4076.0840456  4076.1518297  4076.1871617 km
rsmat eci     0.6392760     0.6240061     0.6160836
rsmat eci     0.4251887     0.4472829     0.4581262
rsmat eci     0.6390713     0.6390819     0.6390874

lmatii =

    0.9298    0.5114    0.2405
    0.0865    0.6246    0.7672
    0.3578    0.5902    0.5946

this should be the inverse of what the code finds later

li =

   26.5482   52.9061  -78.9938
  -72.7755 -152.3382  225.9718
   56.2593  119.3697 -175.0762
```

(2) 이렇게 ECI좌표계를 기준으로 한 벡터값을 구해보았다. 하지만 우리가 최종적으로 구하고자 하는 값은 $\vec{r}$이므로 $\vec{r}_{site}$의 값에 ρL을 더해주어야 한다. (∵ $\vec{r}$ = ρ$\hat{L}$ + $\vec{r}_{site}$)

(3) 이를 위해 $c_1\vec{r}_1 + c_2\vec{r}_2 + c_3\vec{r}_3$ = 0을 만족시키는 c_1, c_2, c_3를 설정하고 $\vec{M}$ (= $\vec{L}^{-1} \times \vec{r}_{site}$)를 계산하면 ρ를 알 수 있다.

위의 과정을 통해 최종적으로 구한 위치벡터 (R_1, R_2, R_3)의 값은 아래와 같다.



2. 위에서 구한 R_2로 Algorithm53 즉, Gibbs method를 이용하여 v_2를 구하는 것이다.

편집기 - C:₩Users₩이진진₩Downloads₩우동2_HW4_codes₩우동2_HW4_code2016_수정₩ex7_2.m

ex7_2.m | site.m | ecef2eci.m

```
286
287         % ---------------------- run the angles-only routine ------------------
288         if typerun == 'l'
289             [r2,v2] = anglesl( decl1,decl2,decl3,rtasc1,rtasc2, ...
290                                rtasc3,jd1,jd2,jd3, rsite1,rsite2,rsite3, re, mu, tu );
291             processtype = 'anglesl';
292         end
293         if typerun == 'd'
294             [r2,v2] = anglesdr( decl1,decl2,decl3,rtasc1,rtasc2, ...
295                                rtasc3,jd1,jd2,jd3, rsite1,rsite2,rsite3, re, mu, tu );
296             processtype = 'anglesdr';
297         end
298         if typerun == 'g'
299            [r2,v2] = anglesg( decl1,decl2,decl3,rtasc1,rtasc2, ...
300                               rtasc3,jd1,jd2,jd3, rsite1,rsite2,rsite3, re, mu, tu );
301             processtype = 'anglesg';
302         end
303         if typerun == 'o'
304            [r2,v2] = anglesgood( decl1,decl2,decl3,rtasc1,rtasc2, ...
305                                  rtasc3,jd1,jd2,jd3, rsite1,rsite2,rsite3 );
306             processtype = 'anglesgood';
307         end
308
```

Line 301에서 볼 수 있듯이 Gauss에서는 processtype을 anglesg로 선정한다. 이 점에서 힌트를 얻어 angleg 파일을 실행해보았고 아래와 같이 v_2에 관한 데이터를 출력하는 코드로 이루어진 파일임을 알 수 있었다.

편집기 - C:₩Users₩이진진₩Downloads₩우동2_HW4_codes₩우동2_HW4_code2016_수정₩anglesg.m

ex7_2.m | site.m | ecef2eci.m | anglesg.m

```
% [r2,v2] = anglesg ( decl1,decl2,decl3,rtasc1,rtasc2,rtasc3,jd1,jd2,jd3,rs1,rs2,rs3, re, mu );
% ---------------------------------------------------------------------------

function [r2, v2] = anglesg ( decl1,decl2,decl3,rtasc1,rtasc2, ...
                    rtasc3,jd1,jd2,jd3, rs1, rs2, rs3, re, mu, tu );

    % ------------------------  implementation  ------------------------
    ddpsi = 0.0;  % delta correctinos not needed for this level of precision
    ddeps = 0.0;
    magr1in = 2.0*re; % initial guesses
    magr2in = 2.01*re;
    direct = 'y';  % direction of motion short way

    % ---------- set middle to 0, find decls to others -----------
    tau1= (jd1-jd2)*tu;  % sec
    tau3= (jd3-jd2)*tu;
    fprintf(1,'jd123 %14.6f %14.6f %14.6f tau %11.7f  %11.7f  \n',jd1,jd2,jd3,tau1,tau3);

    % ---------------  find line of sight unit vectors  ---------------
    l1(1)= cos(decl1)*cos(rtasc1);
    l1(2)= cos(decl1)*sin(rtasc1);
    l1(3)= sin(decl1);

    l2(1)= cos(decl2)*cos(rtasc2);
    l2(2)= cos(decl2)*sin(rtasc2);
    l2(3)= sin(decl2);

    l3(1)= cos(decl3)*cos(rtasc3);
    l3(2)= cos(decl3)*sin(rtasc3);
    l3(3)= sin(decl3);
```

(1) 이 파일을 작동시켜 v_2를 산출시켜야 하는데 이를 위해 수업 시간에 배운 relative motion 내용을 토대로 R_2 위치를 고정시킨 후에 R_1, R_3을 먼저 계산한다. 이 때 R_2의 값은 1번 문제의 결과 값인 ρ_2를 통해 구할 수 있다.

(2) ρ_2를 알기 위해서 $\vec{L}(\hat{L}_1+\hat{L}_2+\hat{L}_3)$을 구한 후에 역행렬과 determinant까지 구하고 이 값들을 d의 정의에 따라 계산하면 d 값을 알 수 있다. ($\rho_2=d_1+d_2$u)

(3) 위에서 구한 값들을 이용해 r에 대한 8차 방정식의 계수를 각각 설정하여 실행하면 R_2를 구할 수 있고, 1-(3)에서 이용한 c_1, c_2, c_3를 차례대로 대입하면 R_1, R_2, R_3를 최종적으로 구하게 된다.

편집기 - C:₩Users₩이진진₩Downloads₩우동2_HW4_codes₩우동2_HW4_code2016_수정₩anglesg.m

ex7_2.m | site.m | ecef2eci.m | anglesg.m

```
236
237         % ---------- now form the three position vectors ----------
238         for i= 1 : 3
239             r1(i)=  rhomat(1,1)*l1eci(i)/c1 + rs1(i);
240             r2(i)= -rhomat(2,1)*l2eci(i)    + rs2(i);
241             r3(i)=  rhomat(3,1)*l3eci(i)/c3 + rs3(i);
242         end
243         magr1 = mag( r1 );
244         magr2 = mag( r2 );
245         magr3 = mag( r3 );
246
247         [v2,theta,theta1,copa,error] = gibbs(r1,r2,r3);
248
249         rad = 180.0/pi;
250         fprintf(1,'r1 %11.7f %11.7f %11.7f %11.7f %11.7f \n',r1,theta*rad,theta1*rad);
251         fprintf(1,'r2 %11.7f %11.7f %11.7f \n',r2);
252         fprintf(1,'r3 %11.7f %11.7f %11.7f \n',r3);
253         fprintf(1,'w gibbs km/s       v2 %11.7f %11.7f %11.7f \n',v2);
254
255         if ( (strcmp(error, '          ok') == 0) && (copa < 1.0/rad) ) % 0 is false
256             [p,a,ecc,incl,omega,argp,nu,m,arglat,truelon,lonper ] = rv2coe (r2,v2, mu);
257             fprintf(1,'coes init ans %11.4f %11.4f %13.9f %13.7f %11.5f %11.5f %11.5f %1
258                 p,a,ecc,incl*rad,omega*rad,argp*rad,nu*rad,m*rad );
259             % --- hgibbs to get middle vector ----
260             [v2,theta,theta1,copa,error] = hgibbs(r1,r2,r3,jd1,jd2,jd3);
261             fprintf(1,'using hgibbs: ' );
262         end
263
264         [p,a,ecc,incl,omega,argp,nu,m,arglat,truelon,lonper ] = rv2coe (r2,v2, mu);
265         fprintf(1,'coes init ans %11.4f %11.4f %13.9f %13.7f %11.5f %11.5f %11.5f %11.5f
266             p,a,ecc,incl*rad,omega*rad,argp*rad,nu*rad,m*rad );
267         %fprintf(1,'dr %11.7f m %11.7f m/s \n',1000*mag(r2-r2ans),1000*mag(v2-v2ans) );
268
269         if ( ll <= 8 )  % 4
270             % --- now get an improved estimate of the f and g series --
271             u= mu / ( magr2*magr2*magr2 );
272             rdot= dot(r2,v2)/magr2;
273             udot= (-3.0*mu*rdot) / (magr2^4);
```

(4) angleg 파일의 237line부터는 gibbs와 herrick-gibbs 방법을 통해 위치벡터 R_1, R_2, R_3를 가지고 v_2를 구하는 코드로 이루어져있음을 확인할 수 있고 v_2를 구하기 위해 f, g 함수를 이용해야 한다는 것이 언급되어 있다.

269line 부터는 이 두 함수의 계산식이 나와 있고 이 곳에 데이터들을 대입하여

$v_2 = \dfrac{R_3 - fR_2}{g}$ 를 통해 v_2를 최종적으로 구하게 된다.

이렇게 계산해본 문제 2번의 결과는 아래와 같다.

```
명령 창
solution by anglesg

r2        0.9416955   0.8352886   1.0057268 er    6006.2628584  5327.5849219  6414.6630493 km
r2 ans    0.9966054   0.8294792   1.0208932 er    6356.4860340  5290.5322568  6511.3969786 km
v2       -0.5462280   0.6017473   0.2068567 er/tu  -4.3181324    4.7570326    1.6352781 km/s
v2 ans   -0.5278628   0.6042162   0.2176080 er/tu  -4.1729482    4.7765497    1.7202709 km/s
          p km        a km        ecc        incl deg     raan deg     argp deg     nu deg     m deg
coes  11398.4092  11854.1308  0.196071716  40.0764920   329.84020     19.66012    56.16500   39.00257
          p km        a km        ecc        incl deg     raan deg     argp deg     nu deg     m deg
coes  11756.1821  12246.0230  0.200000000  40.0000073   329.99999     20.00001    54.23598   37.22711
fx
```

Q2

3. 마지막으로 1, 2번에서 구한 R_2와 v_2를 이용해 6가지 궤도요소를 결정하는 것이다. 이를 위해 Algorithm9인 RV2COE를 이용했다.

ALGORITHM 9: RV2COE ($\vec{r}_{IJK}, \vec{v}_{IJK} \Rightarrow p, a, e, i, \Omega, \omega, \nu(u, \lambda_{true}, \tilde{\omega}_{true})$)

$$\vec{h} = \vec{r} \times \vec{v} \qquad h = |\vec{h}|$$

$$\vec{n} = \hat{K} \times \vec{h}$$

$$\vec{e} = \frac{\left(v^2 - \frac{\mu}{r}\right)\vec{r} - (\vec{r} \cdot \vec{v})\vec{v}}{\mu} \qquad e = |\vec{e}|$$

$$\xi = \frac{v^2}{2} - \frac{\mu}{r}$$

IF $e \neq 1.0$ THEN

$$a = -\frac{\mu}{2\xi}$$

$$p = a(1 - e^2)$$

ELSE

$$p = \frac{h^2}{\mu} \text{ and } a = \infty$$

$$\cos(i) = \frac{h_K}{|\vec{h}|}$$

$$\cos(\Omega) = \frac{n_I}{|\vec{n}|} \qquad \text{IF}(n_J < 0)\text{THEN } \Omega = 360° - \Omega$$

$$\cos(\omega) = \frac{\vec{n} \cdot \vec{e}}{|\vec{n}||\vec{e}|} \qquad \text{IF}(e_K < 0)\text{THEN } \omega = 360° - \omega$$

$$\cos(\nu) = \frac{\vec{e} \cdot \vec{r}}{|\vec{e}||\vec{r}|} \qquad \text{IF}(\vec{r} \cdot \vec{v} < 0)\text{THEN } \nu = 360° - \nu$$

Special Cases

Elliptical equatorial:

$$\cos(\tilde{\omega}_{true}) = \frac{e_I}{|\vec{e}|} \qquad \text{IF}(e_J < 0)\text{THEN } \tilde{\omega}_{true} = 360° - \tilde{\omega}_{true}$$

Circular Inclined:

$$\cos(u) = \frac{\vec{n} \cdot \vec{r}}{|\vec{n}||\vec{r}|} \qquad \text{IF}(r_K < 0)\text{THEN } u = 360° - u$$

Circular equatorial:

$$\cos(\lambda_{true}) = \frac{r_I}{|\vec{r}|} \qquad \text{IF}(r_J < 0)\text{THEN } \lambda_{true} = 360° - \lambda_{true}$$

직접 계산한다면 specific angular momentum $\vec{h}$와 specific energy ξ 등 복잡한 수식을 이용하여 6가지의 궤도요소를 결정할 수 있다.

편집기 - C:₩Users₩이진진₩Downloads₩우동2_HW4_codes₩우동2_HW4_code2016_수정₩ex7_2.m

ex7_2.m | site.m | ecef2eci.m | anglesg.m

```
136
137             % at 8-20-12 11:48:28.000 center time,
138             r2ans = [6356.48603405     5290.53225676     6511.39697857   ];
139             v2ans = [  -4.17294816     4.77654968     1.72027091];
140             [p,a,ecc,incl,omega,argp,nu,m,arglat,truelon,lonper ] = rv2coe (r2ans,v2ans,  mu);
141
142             year  =  2012;
143             mon   =   8;
144             day   =  20;
145             hr    =  11;
146             min   =  55;
147             sec   =  28.0000;
148             dut1  =  -0.6096413;
149             dat   = 35; % leap second in July 2012
150             xp    =  0.137495 * convrt;
151             yp    =  0.342416 * convrt;
152             lod   =  0.0;
153             timezone= 0;
154             terms = 0;
155             ddpsi = 0.0;
156             ddeps = 0.0;
```

직접 계산하는 방법 외에 코드를 활용한다면 ex7_2 파일의 140line에 해당하는 내용을 통해 보다 손쉽게 6가지의 궤도요소를 결정할 수 있다. 문제 1, 2번에서 구한 R_2와 v_2를 RV2COE 파일로 계산할 수 있다. 프로그램을 이용하여 계산한 결과 6개의 궤도요소 값은 아래와 같다.

명령 창

```
solution by anglesg

r2        0.9416955    0.8352886    1.0057268 er     6006.2628584  5327.5849219  6414.6630493 km
r2 ans    0.9966054    0.8294792    1.0208932 er     6356.4860340  5290.5322568  6511.3969786 km
v2       -0.5462280    0.6017473    0.2068567 er/tu  -4.3181324     4.7570326    1.6352781 km/s
v2 ans   -0.5278628    0.6042162    0.2176080 er/tu  -4.1729482     4.7765497    1.7202709 km/s
          p km          a km         ecc        incl deg     raan deg     argp deg     nu deg      m deg
coes   11398.4092   11854.1308   0.196071716   40.0764920   329.84020     19.66012    56.16500    39.00257
          p km          a km         ecc        incl deg     raan deg     argp deg     nu deg      m deg
coes   11756.1821   12246.0230   0.200000000   40.0000073   329.99999     20.00001    54.23598    37.22711
```

Q3